

Adaptive Switching Strategies for the Trisection–False Position Hybrid

CSE402 Numerical Analysis, Simulation and Modeling Sessional

Sheikh Iftikharun Nisa

Student ID: 2105069

Department of Computer Science and
Engineering

Bangladesh University of Engineering
and Technology

Tanzim Tousif

Student ID: 2105071

Department of Computer Science and
Engineering

Bangladesh University of Engineering
and Technology

Sadia Tasnim

Student ID: 2105077

Department of Computer Science and
Engineering

Bangladesh University of Engineering
and Technology

Mahbuba Sayed Sinthia

Student ID: 2105078

Department of Computer Science and
Engineering

Bangladesh University of Engineering
and Technology

Dibya Jyoti Sarker

Student ID: 2105084

Department of Computer Science and
Engineering

Bangladesh University of Engineering
and Technology

Abstract

Hybrid bracketing root finders are usually compared by iteration count. Badr, Almotairi and El Ghamry proposed a static trisection–false position hybrid (TF) that needs 98 total iterations on 15 benchmark equations, against 121 for the bisection–false position hybrid (BF). Each TF iteration, however, may spend up to three new function evaluations regardless of how the function behaves. We ask whether an adaptive selection rule reduces the actual work, and which runtime signal should drive the switch. We implement ten methods on one shared bracket engine in which every distinct function evaluation is cached and charged: five classical methods, the two static hybrids, and three adaptive selectors that switch on the residual ratio, on log-progress per function call, or on endpoint-reuse counters. We reproduce 28 of the 30 published per-problem iteration counts and evaluate all methods on the original 15 equations and on five additional stress equations, verifying every root against 80-digit mpmath references. At a residual tolerance of 10^{-14} , TF uses the fewest iterations among the bracketed methods (99) but 325 function calls, while BF needs only 273. The stagnation-aware policy is the strongest adaptive variant: 300 calls (7.7% fewer than TF), 11 of 15 head-to-head wins against TF, and no equation above 27 calls. No adaptive policy beats BF in aggregate at 10^{-10} or 10^{-14} ; stagnation-aware is the cheapest method only at 10^{-6} . Iteration count is therefore an unreliable proxy for computational cost.

CCS Concepts

• **Mathematics of computing** → **Nonlinear equations**; *Numerical analysis*.

Keywords

root finding, bracketing methods, false position, trisection, hybrid algorithms, adaptive switching, function evaluations

1 Introduction and Base Paper Review

1.1 Background

Finding a root of a continuous scalar function f on an interval $[a, b]$ with $f(a)f(b) < 0$ is one of the oldest problems in numerical analysis [4]. Bracketing methods keep a sign change at every step and are therefore guaranteed to converge. Bisection halves the bracket and converges linearly with a fixed rate; false position (regula falsi) replaces the midpoint by the secant intercept and is often much faster, but it can stall when one endpoint is retained for many consecutive steps, which happens when f is convex or concave on the bracket. Classical remedies modify the retained function value (the Illinois method [6] and the Anderson–Björck method [1]) or combine interpolation with bisection safeguards (Dekker–Brent [3], Chandrupatla [5]). Open methods such as Newton and secant converge faster locally but give no bracketing guarantee.

A simpler family of hybrids combines an interval-reduction step with a false position step inside every iteration. Sabharwal [8] blended bisection with false position (BF), and Badr et al. [2] replaced bisection by trisection.

1.2 Base Paper

Badr, Almotairi and El Ghamry [2] proposed the static trisection–false position hybrid TF. Every iteration evaluates both trisection points

$$x_1 = a + \frac{b-a}{3}, \quad x_2 = b - \frac{b-a}{3}, \quad (1)$$

and the false position point

$$x_F = \frac{a f(b) - b f(a)}{f(b) - f(a)}, \quad (2)$$

keeps the candidate with the smallest $|f(x)|$ as the current approximation, and narrows the bracket to a sub-interval that still contains a sign change. On 15 benchmark equations (their Table 1), the paper reports 98 total iterations for TF against 121 for BF, and concludes that TF is the better hybrid.

The comparison has a structural gap. Iterations were compared, but a TF iteration pays up to three new function evaluations while a BF iteration pays two, and both pay this cost whatever the function is doing. When f is expensive, as it usually is when root finding matters, the relevant cost is the number of function evaluations, not the number of iterations.

1.3 Objective

Our research question is: *does adaptivity actually pay, and which signal is the right one to adapt on?* We keep the bracket engine of the hybrids fixed and replace only the rule that selects which candidate points are evaluated in a step. We study three adaptive policies, each driven by a different signal:

- **Adaptive threshold**, which adapts on the residual ratio;
- **Cost-aware**, which adapts on progress per function call;
- **Stagnation-aware**, which adapts on endpoint reuse.

For every method we measure iterations, function calls, wall-clock time and success. Our contributions are (i) a reproduction of the base paper with a documented list of discrepancies, (ii) a common engine with fair cost accounting for ten methods, (iii) a comparison of the three adaptive policies against both static hybrids, and (iv) a five-equation stress suite that probes failure modes not covered by the original benchmark.

2 Methodology

2.1 Classical Methods

We implement five classical methods as baselines. *Bisection* evaluates the midpoint $m = (a + b)/2$ and keeps the half with a sign change. *Trisection* evaluates both points in Eq. (1) and keeps the third with a sign change. *False position* evaluates x_F from Eq. (2). *Newton's method* iterates $x_{k+1} = x_k - f(x_k)/f'(x_k)$, starting from the left endpoint a . The *secant method* replaces f' by a finite-difference slope through the two most recent iterates. Newton and secant are implemented in their classical, unsafeguarded form: there is no clipping to $[a, b]$ and no bracketed fallback, so that their failure modes remain visible.

2.2 Adaptive Policy 1: Adaptive Threshold

This policy compares the progress of a tentative false position step with recent progress. With $r = |f_{\text{new}}|/|f_{\text{old}}|$ the residual ratio of a step, it keeps a threshold

$$\tau_k = \frac{1}{3} \sum_{j=k-3}^{k-1} r_j, \quad \tau = 1 \text{ before three ratios exist.} \quad (3)$$

In each step it first probes x_F . If the probe ratio satisfies $r_F < \tau$, the false position step is accepted at a cost of one new call. Otherwise the policy trisects, and the step costs three new calls, because the rejected probe is still charged. The window uses only past ratios, so no future information enters the decision.

2.3 Adaptive Policy 2: Cost-Aware

This policy measures the log-progress per function call of each step type,

$$E = \frac{\ln e_{\text{old}} - \ln e_{\text{new}}}{\text{new calls in the step}}, \quad (4)$$

where e is the error measure tracked by the engine. It keeps a three-step running mean of E for false position (one call per step) and for trisection (two calls per step) and selects the step type with the higher mean. A bootstrap performs one false position step and then one trisection step so that both estimates exist. The policy is greedy: it never explores, so the estimate for a step type that is no longer used is never refreshed and can go stale.

2.4 Adaptive Policy 3: Stagnation-Aware False Position

This policy targets the known failure mode of false position directly. It keeps two counters that record how many consecutive steps the left and the right endpoint have been reused, and resets a counter when its endpoint moves. If either counter reaches the reuse limit L (default $L = 3$), the policy forces a trisection step at a cost of two calls; otherwise it takes a false position step at a cost of one call. The counters track real endpoint reuse as reported by the engine through OBSERVE. We also test $L = 2$.

2.5 Cost Accounting

We measure every method in four ways: iterations, function calls, wall-clock time and success. The accounting rules are fixed in advance.

- Every distinct $f(x)$ call counts, including the two initial endpoint evaluations and rejected probes. A cached point is never charged twice.
- Failures remain in the totals. Pairwise comparisons use only problems on which both methods succeed inside the interval.
- Newton's f' calls are reported separately and are not folded into the f -call rankings.

3 Implementation and Experiments

3.1 Software

The implementation is written in Python. `main.py` is used for running the program. `experiment.py` runs the benchmark, records timings, and writes per-method results, pairwise comparisons, convergence histories and plots. `solvers.py` file has the implementation of all methods.

3.2 Parameters

Table 1 lists the experimental parameters. The default residual tolerance is $\varepsilon = 10^{-14}$ with a cap of 1,000 iterations. Each method-problem pair is run once as a warm-up and then ten times with timing, and the order of methods is shuffled with seed 402 to avoid systematic ordering effects. The sensitivity study repeats the benchmark at $\varepsilon \in \{10^{-6}, 10^{-10}, 10^{-14}\}$ and with reuse limit $L \in \{2, 3\}$.

3.3 Benchmarks

Base-paper benchmark. We use the 15 equations of Table 1 of the base paper [2] with the same intervals; they are listed with their results in Table 4. The set consists of polynomials (P1–P6, P11), mixed exponential and trigonometric functions (P7–P10, P12–P15), and one high-degree polynomial, $x^{10} - 1$ (P11), that is very flat near the left end of its bracket.

Table 1: Experimental parameters.

Parameter	Value
Stopping rule	$ f(x) \leq \varepsilon$ only
Default tolerance ε	10^{-14}
Tolerances in sensitivity study	10^{-6} , 10^{-10} , 10^{-14}
Iteration cap $N_{\max}$	1,000
Timing	1 warm-up + 10 timed runs
Method order	shuffled, seed 402
Threshold window	last 3 ratios, $\tau_0 = 1$
Cost-aware window	3-step mean of E
Stagnation reuse limit L	3 (default), 2

Table 2: Stress suite (not part of the base paper).

Equation	Failure mode probed
$e^x - 1000$	Absolute residual below binary64 resolution: the best attainable residual exceeds ε .
$x^{20} - 1$	Extreme convexity: false position retains one endpoint for many steps.
$(x - 0.3)^3$	Triple root: f and f' vanish together and the function is flat at the root.
$10^{-12} * (x^2 - 2)$	Residual tolerance is not invariant under scaling of f .

Stress suite. The original benchmark contains no case on which a bracketed hybrid is close to failure, so we add five stress equations that are not in the base paper. Each probes one failure mode, as summarised in Table 2.

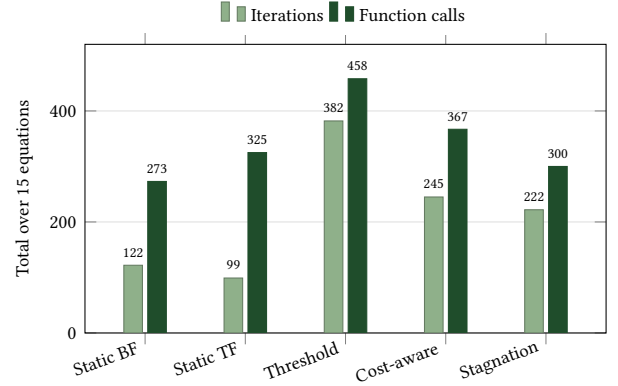
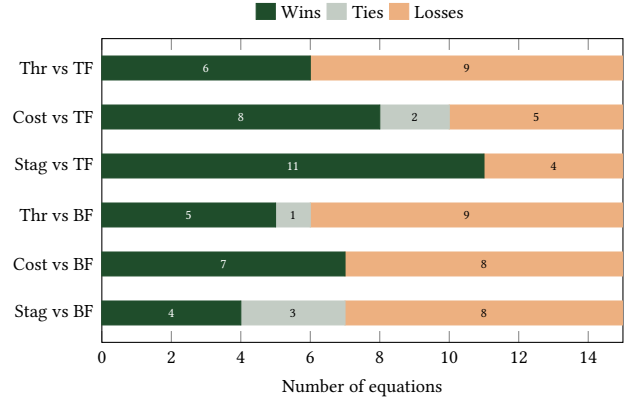
4 Results and Discussion

4.1 Reproduction of the Base Paper

Our implementation gives 122 total iterations for static BF (paper: 121) and 99 for static TF (paper: 98). Of the 30 per-problem iteration counts (15 equations, two hybrids), 28 match the paper exactly. The only difference is on P3 ($x^2 - 10$), where both hybrids need one extra iteration; the most likely cause is a different floating-point evaluation order in the false position formula (2). We report published and measured values side by side and do not force a match.

During the reproduction we found four issues in the base paper:

- (1) Some printed residuals are labelled 0, but they are not zero when the displayed root is substituted back into f .
- (2) The text around Figures 4–5 attributes the behaviour of P11 to P4. P4’s root in $[1, 4]$ is $x = 2$, which trisection finds in a single step.
- (3) The “error” column of Table 2 is the difference between successive iterates, while the algorithms stop on $|f(x)|$.
- (4) The $\log_2$ and $\log_3$ step bounds limit the interval width, not the residual stopping criterion that is actually used.

**Figure 1: Total iterations and function calls of the five hybrids on the 15 base-paper equations at $\varepsilon = 10^{-14}$.****Figure 2: Per-equation wins, ties and losses in function calls of each adaptive policy against static TF and static BF.**

4.2 Fewer Iterations Do Not Mean Less Work

Figure 1 compares total iterations and function calls for the five hybrids at $\varepsilon = 10^{-14}$. Static TF takes the fewest iterations (99) but spends 325 calls. Static BF is the cheapest hybrid with 273 calls, and no adaptive policy beats it in aggregate. Among the adaptive policies, stagnation-aware is the best with 300 calls, 7.7% fewer than static TF. Adaptive threshold is the worst with 458 calls, of which 98 come from P11 alone. The ranking by iterations and the ranking by function calls disagree for every adaptive policy, which is the central point of this study.

4.3 Head-to-Head Comparison

Figure 2 counts per-equation wins, ties and losses in function calls of each policy against each static hybrid, and Table 3 gives the median per-equation call ratio. Two results stand out. First, adaptivity beats static TF: stagnation-aware wins 11 of 15 equations and cost-aware wins 8 of 15, with median ratios of 0.885 and 0.824. Second, adaptivity does not beat static BF: every policy loses more equations than it wins against BF, and every median ratio against BF is above 1.

Table 3: Median per-equation function-call ratio (policy / static hybrid). Values below 1 favour the policy.

Policy	vs static TF	vs static BF
Adaptive threshold	1.118	1.250
Cost-aware	0.824	1.150
Stagnation-aware	0.885	1.038

4.4 Per-Equation Cost

Table 4 lists the function calls of each hybrid on each equation. There is no single winner: every hybrid is cheapest, or tied for cheapest, on at least one equation.

Adaptive threshold. P2, P4, P6, P11, P13 and P14 each cost between 37 and 98 calls. The residual ratio is the wrong signal for endpoint locking. When false position stagnates, each step still produces a steady, small improvement in $|f|$; the ratio r_f stays below a threshold that is itself an average of such ratios, so the slow steps keep being accepted. On P11 this costs 98 calls.

Cost-aware. The two outliers, P6 (59 calls) and P10 (61 calls), have the same cause. After a single false position step, the policy judges trisection to be more productive per call and then trisects 28 and 29 times in a row. Because the policy is greedy, the stale false position estimate is never revisited. The call counts are consistent with this explanation: $2 + 1 + 28 \cdot 2 = 59$ and $2 + 1 + 29 \cdot 2 = 61$. This is why cost-aware has the best median ratio against TF (0.824) yet the second-worst total.

Stagnation-aware. This policy never blows up. Its worst case is 27 calls on P11, the equation on which plain false position stagnates most severely and the failure mode the policy was designed for. It pays for this robustness on equations where false position converges quickly anyway; for example, on P4 it needs 24 calls while TF needs 5.

4.5 All Ten Methods

Figure 3 shows the iterations and function calls of all ten methods per equation. Bisection and trisection are nearly flat: their step count is governed by how many reductions of the interval are needed before the residual falls below ϵ , which depends mainly on the initial width and only weakly (logarithmically) on the slope of f at the root. False position and adaptive threshold spike on P11. Stagnation-aware stays within 10–27 calls on every equation, while plain false position ranges from 8 to 141.

Table 5 summarises all methods. The open methods use the fewest calls when they succeed, but neither solves all 15 equations. Newton’s method fails on P6, P9 and P11, where $f'(a) = 0$ at the left endpoint, and on P10 it converges to a valid root outside $[-2, 4]$. The secant method stalls on P11, where $x^{10} - 1$ is flat near 0. Every bracketed method solves all 15 equations.

4.6 Sensitivity to Tolerance and Reuse Limit

Figure 4 shows how the totals change with the tolerance. The ranking depends on ϵ . At $\epsilon = 10^{-6}$ stagnation-aware is the cheapest method overall (189 calls against 196 for BF); at 10^{-10} and 10^{-14}

BF pulls ahead. A plausible explanation is that BF pays two calls per step but halves the bracket every time, so its advantage grows as more digits are demanded, while the stagnation-aware policy spends its first steps on cheap false position progress that matters most at loose tolerances. All three policies solve all 15 equations at every tested tolerance without per-equation tuning.

Lowering the reuse limit from $L = 3$ to $L = 2$ reduces the stagnation-aware iterations from 222 to 208 but leaves the total at 300 calls: fewer iterations, the same work. This is a second, internal example of the main finding.

4.7 Stress Suite

The stress equations expose limits that the base benchmark hides. For $e^x - 1000$, the best binary64 residual near the root $\ln 1000$ is about 2.3×10^{-13} , larger than $\epsilon = 10^{-14}$: since $f'(x) \approx 1000$ there, one unit in the last place of x already changes f by more than ϵ , and the value 1000 itself carries a rounding error of order 10^{-13} . Every bracketed method correctly reports a precision limit instead of claiming convergence. On $x^{20} - 1$ and $(x - 0.3)^3$, plain false position hits the 1,000-iteration cap, while all hybrids succeed; this is the stagnation failure that the hybrids are designed to prevent. On the flat cubic and the scaled quadratic, a tiny residual does not guarantee a tiny root error, so for these cases we report the true error against the 80-digit references as well [7].

4.8 Discussion and Threats to Validity

The function-call metric assumes that evaluating f dominates the cost. For the cheap benchmark functions used here, interpreter overhead and the bookkeeping of the adaptive selectors are comparable to the cost of a call, so wall-clock rankings may differ from call rankings; the call counts should be read as a model of expensive functions. The benchmark is small, and many per-equation wins differ by one or two calls, so the win counts in Figure 2 are descriptive and carry no statistical significance. Finally, the absolute residual criterion $|f(x)| \leq \epsilon$ is not scale-invariant, as the scaled-quadratic and $e^x - 1000$ cases show; conclusions at a fixed ϵ depend on the scale of each f .

5 Conclusion and Future Work

Endpoint stagnation is the right signal to adapt on, but adaptivity is not an unconditional win: at tight tolerances, static BF is still cheaper. Our four main findings are:

- (1) **Stagnation-aware is the best adaptive policy.** It needs 300 calls against 325 for static TF (−7.7%), wins 11 of 15 equations against TF, and is never catastrophic; its worst case is 27 calls.
- (2) **Residual-ratio thresholds miss endpoint locking.** Steady, small improvements keep passing the threshold, and on P11 the threshold policy needs 98 calls.
- (3) **Cost-aware looks good per equation but is worse in total.** Its median ratio against TF is 0.82, but its greedy estimates go stale on P6 and P10.
- (4) **Count work, not iterations.** Static TF has the fewest iterations yet costs more than BF, and reuse limit 2 cuts iterations but not calls.

Table 4: Function calls per equation at $\varepsilon = 10^{-14}$ (lower is better). Bold marks the cheapest hybrid on each equation.

ID	$f(x)$	Static BF	Static TF	Threshold	Cost-aware	Stagnation
P1	$x^2 - 3$	18	22	16	13	18
P2	$x^2 - 5$	22	26	43	28	25
P3	$x^2 - 10$	18	23	18	15	18
P4	$x^2 - x - 2$	6	5	37	20	24
P5	$x^2 + 2x - 7$	11	22	24	16	19
P6	$x^3 - 2$	20	26	37	59	23
P7	$xe^x - 7$	24	23	30	16	20
P8	$x - \cos x$	18	23	15	12	15
P9	$x \sin x - 1$	14	17	10	10	10
P10	$x \cos x + 1$	22	26	14	61	17
P11	$x^{10} - 1$	26	29	98	44	27
P12	$x^2 + e^{x/2} - 5$	18	20	17	13	18
P13	$\sin x \sinh x + 1$	20	23	40	23	24
P14	$e^x - 3x - 2$	20	23	40	23	24
P15	$\sin x - x^2$	16	17	19	14	18
Total		273	325	458	367	300

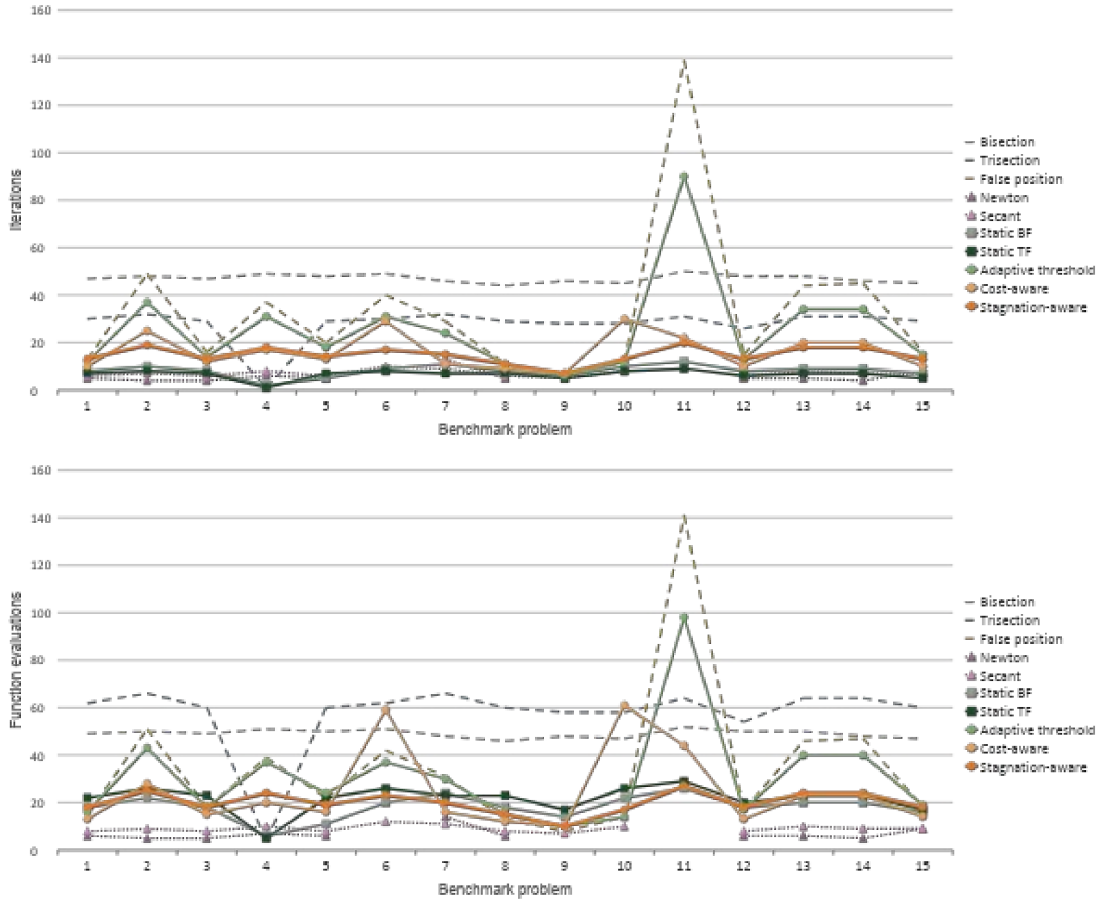
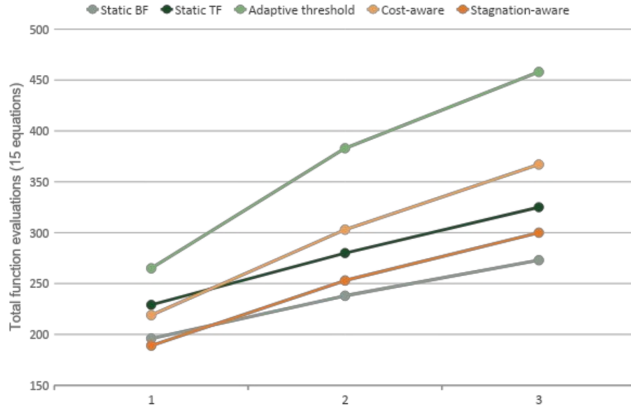
**Figure 3: Iterations (top) and function calls (bottom) of all ten methods on the 15 base-paper equations at $\varepsilon = 10^{-14}$.**

Table 5: All methods on the 15 base-paper equations at $\varepsilon = 10^{-14}$. Newton’s f' calls are listed separately.

Method	Solved	Iter.	f calls	f' calls
Bisection	15/15	706	736	—
Trisection	15/15	416	862	—
False position	15/15	492	522	—
Newton	12/15	80	92	80
Secant	14/15	104	133	—
Static BF	15/15	122	273	—
Static TF	15/15	99	325	—
Adaptive threshold	15/15	382	458	—
Cost-aware	15/15	245	367	—
Stagnation-aware	15/15	222	300	—

**Figure 4: Total function calls over the 15 equations against tolerance. Positions 1, 2 and 3 on the horizontal axis correspond to $\varepsilon = 10^{-6}$, 10^{-10} and 10^{-14} .**

Limitations. The study uses 15 benchmark equations and five stress equations, which is too few for statistically supported rankings. We compare only against BF, TF and unsafeguarded open methods. All policy parameters (window length 3, reuse limit 2 or 3) were fixed by hand, and the stopping rule is an absolute residual.

Future work. Natural next steps are (i) giving the cost-aware policy an exploration mechanism, for example periodic re-sampling or an upper-confidence-bound rule, so that stale estimates are re-freshed; (ii) combining the stagnation signal with a bisection rather than a trisection fallback, since BF’s two-call step is cheaper; (iii) evaluating on a larger, randomly generated benchmark with intervals that do not place roots on trisection or bisection points.

6 Data and Code Reproducibility

The github repository maintained for this project is public. The URL link for the repository: https://github.com/tasnimsadia013/CSE_402_Project

Acknowledgments

We thank the instructors and our esteemed supervisor, Md. Emamul Haque Pranta Sir of CSE402 Numerical Analysis, Simulation and Modeling Sessional for their guidance throughout the project and inspiring us for future works of this project.

References

- [1] Ned Anderson and Åke Björck. 1973. A New High Order Method of Regula Falsi Type for Computing a Root of an Equation. *BIT Numerical Mathematics* 13, 3 (1973), 253–264. doi:10.1007/BF01951936
- [2] Elsayed Badr, Sultan Almotairi, and Abdallah El Ghamry. 2021. A Comparative Study among New Hybrid Root Finding Algorithms and Traditional Methods. *Mathematics* 9, 11 (2021), 1306. doi:10.3390/math9111306
- [3] Richard P. Brent. 1973. *Algorithms for Minimization without Derivatives*. Prentice-Hall, Englewood Cliffs, NJ.
- [4] Richard L. Burden, J. Douglas Faires, and Annette M. Burden. 2016. *Numerical Analysis* (10th ed.). Cengage Learning, Boston, MA.
- [5] Tirupathi R. Chandrupatla. 1997. A New Hybrid Quadratic/Bisection Algorithm for Finding the Zero of a Nonlinear Function without Using Derivatives. *Advances in Engineering Software* 28, 3 (1997), 145–149. doi:10.1016/S0965-9978(96)00051-8
- [6] M. Dowell and P. Jarratt. 1971. A Modified Regula Falsi Method for Computing the Root of an Equation. *BIT Numerical Mathematics* 11, 2 (1971), 168–174. doi:10.1007/BF01934364
- [7] Nicholas J. Higham. 2002. *Accuracy and Stability of Numerical Algorithms* (2nd ed.). SIAM, Philadelphia, PA. doi:10.1137/1.9780898718027
- [8] Chaman Lal Sabharwal. 2019. Blended Root Finding Algorithm Outperforms Bisection and Regula Falsi Algorithms. *Mathematics* 7, 11 (2019), 1118. doi:10.3390/math7111118